

1. Paso de Actividad a Actividad (Sin argumentos)

Enunciado: Navegar de una pantalla a otra de forma simple mediante un Intent.

Kotlin

```
// En la Actividad de origen (clic de un botón)
val intent = Intent(this, MainActivity2::class.java)
startActivity(intent)
```

2. Paso de Actividad a Actividad (Con argumentos)

Enunciado: Enviar datos (como un String) de una actividad a otra usando "extras".

Kotlin

```
// En la Actividad 1 (Envío)
val intent = Intent(this, Activity2Main::class.java)
intent.putExtra("nombre", "Eli")
startActivity(intent)
```

```
// En la Actividad 2 (Recepción)
// 'intent' ya existe por herencia, no se declara
val nombre = intent.getStringExtra("nombre")
```

3. Paso de Actividad a Fragment (Sin argumentos)

Enunciado: Cargar un fragmento dentro de un contenedor (como un FrameLayout) en una actividad.

Kotlin

```
val fragment = MiFragmento()
val transaction = supportFragmentManager.beginTransaction()
transaction.replace(R.id.contenedorPrincipal, fragment)
transaction.commit() // ¡Imprescindible para ejecutar el cambio!
```

4. Paso de Actividad a Fragment (Con argumentos)

Enunciado: Enviar datos a un fragmento usando un "Bundle" antes de realizar la transacción.

Kotlin

```
// En la Actividad (Envío)

val fragment = FragmentA()

val bundle = Bundle()

bundle.putString("nombre", "Eli")

bundle.putInt("edad", 43)


fragment.arguments = bundle // Metemos la "maleta" en el fragmento


supportFragmentManager.beginTransaction().apply {

    replace(R.id.contenedorPrincipal, fragment)

    commit()

}
```

```
// En el Fragmento (Recepción)

// 'arguments' es la propiedad que contiene el Bundle enviado

val nombre = arguments?.getString("nombre")

val edad = arguments?.getInt("edad") ?: 0
```

```
val fragment=FragmentA()

val transaction=supportManger.beginTransaction()

val bundle=Bundle()

bundle.putString("nombre","Eli")

fragments.arguments=bundle

transaction.replace(R.id.ppal,fragment)

transaction.commit
```

5. Actividad B -> A (Retorno de datos con Launcher)

Este es el método para que A reciba un nombre desde B y lo use:

En Actividad A:

Kotlin

```

class ActividadA : AppCompatActivity() {

    // 1. Variable global para usar el dato en toda la clase
    private var nombreUsuario: String? = null

    // 2. La "Aduana" que recibe y USA el dato
    private val miLauncher =
registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { res ->

    if (res.resultCode == RESULT_OK) {

        // SACAMOS el dato del paquete
        val nombre = res.data?.getStringExtra("CLAVE_NOMBRE")

        // USAMOS el dato (Ej: ponerlo en un TextView)
        findViewById<TextView>(R.id.tvSaludo).text = "Hola, $nombre"

        // GUARDAMOS en la variable global para después
        nombreUsuario = nombre
    }
}

// Para lanzar B: miLauncher.launch(Intent(this, ActividadB::class.java))
}

```

En Actividad B:

Kotlin

// Al confirmar:

```

val i = Intent().putExtra("CLAVE_NOMBRE", "Eli")
setResult(RESULT_OK, i)
finish()

```

6. Fragmento → Actividad (Sin argumentos)

Enunciado: El fragmento necesita que la Actividad haga algo (como cerrarse) o llamar a una función sin pasarle datos.

Kotlin

```
// Dentro del Fragmento (ej. al pulsar un botón)
```

```
binding.btnFinalizar.setOnClickListener {
```

```
    // 1. Finalizar la actividad padre directamente
```

```
    requireActivity().finish()
```

```
    // 2. O llamar a una función de la actividad
```

```
    (requireActivity() as MainActivity).hacerAlgo()
```

```
}
```

7. Fragmento → Actividad (Con argumentos)

Enunciado: El fragmento envía un dato (un nombre) a la Actividad para que esta lo guarde y lo use.

A. Forma Directa (Propiedad Pública - *Menos recomendada*)

Kotlin

```
// En la Actividad
```

```
var nombreRecibido: String = ""
```

```
// En el Fragmento
```

```
(requireActivity() as MainActivity).nombreRecibido = "Eli"
```

B. Forma Segura (Mediante Método - *Recomendada*)

En la Actividad (Destino):

Kotlin

```
class MainActivity : AppCompatActivity() {
```

```
    private var nombre: String = "" // Privada para mayor seguridad
```

```
    fun recibirDatosDelFragmento(texto: String) {
```

```

        this.nombre = texto

        // Aquí puedes aprovechar para actualizar un TextView, por ejemplo:
        findViewById<TextView>(R.id.tvTitulo).text = "Hola, $nombre"
    }
}

```

En el Fragmento (Origen):

Kotlin

```

binding.btnEnviar.setOnClickListener {
    val nombreAEnviar = binding.etNombre.text.toString()

    // Llamamos al método de la actividad pasando el argumento
    (requireActivity() as MainActivity).recibirDatosDelFragmento(nombreAEnviar)
}

```

8. Fragmento A → Fragmento B (Sin Argumentos)

Concepto: Reemplazar un fragmento por otro (hermanos) en el mismo contenedor de la actividad.

Kotlin

```

// Dentro del Fragmento A
binding.btnIrB.setOnClickListener {
    // 1. Instanciamos el nuevo fragmento
    val fragmentoB = FragmentoB()

    // 2. Pedimos el manager a la actividad (nuestro padre)
    val manager = parentFragmentManager

    // 3. Iniciamos la transacción

```

```
val transaction = manager.beginTransaction()

// 4. Reemplazamos el contenedor por el nuevo fragmento
transaction.replace(R.id.contenedor_principal, fragmentoB)

// 5. Opcional: Permitir volver atrás con el botón físico del móvil
transaction.addToBackStack(null)

// 6. Confirmamos el cambio
transaction.commit()
}
```

9. Fragmento A → Fragmento B (Con Argumentos)

Concepto: Pasar datos (como un nombre) de un fragmento a otro usando una "maleta" (Bundle).

Kotlin

```
// Dentro del Fragmento A
binding.btnEnviarDato.setOnClickListener {
    val fragmentoB = FragmentoB()

    // 1. Creamos la "maleta" (el Bundle)
    val bundle = Bundle()
    bundle.putString("CLAVE_NOMBRE", "Eli") // Metemos el dato

    // 2. Le pegamos la maleta al fragmento de destino
    fragmentoB.arguments = bundle

    // 3. Hacemos la transacción normal
```

```
val transaction = parentFragmentManager.beginTransaction()

transaction.replace(R.id.contenedor_principal, fragmentoB)

transaction.addToBackStack(null)

transaction.commit()

}
```

```
// --- DENTRO DEL FRAGMENTO B (Recepción) ---
```

```
// En el onCreateView:
```

```
val nombre = arguments?.getString("CLAVE_NOMBRE")
```

Explicación del childFragmentManager

Para que no se te olvide, piensa en esto como una **escalera**:

- **parentFragmentManager (El Ascensor)**: Lo usas cuando quieres moverte entre fragmentos que están en el mismo nivel (como del Fragmento Detalle al Fragmento Modificar). Le pides a la Actividad que te cambie por otro.
- **childFragmentManager (La Escalera de caracol)**: Lo usas solo cuando un fragmento tiene **otros fragmentos viviendo dentro de él**.
 - *Ejemplo*: Tienes un FragmentoPerfil y dentro de su propio diseño XML has puesto un hueco (FrameLayout) para mostrar un FragmentoEstadisticas. Como el "Estadisticas" es un hijo del "Perfil", usas el childFragmentManager.

Resumen jerárquico:

1. **Actividad** gestiona fragmentos principales → supportFragmentManager.
2. **Fragmento** gestiona a sus hermanos → parentFragmentManager.
3. **Fragmento** gestiona a sus hijos internos → childFragmentManager.